

Final Project Description

In this final project, you will implement cryptographic primitives **entirely from scratch** on the **GPU**, benchmark them against CPU equivalents, and analyze the performance-security trade-offs. You will choose one of the projects outlined in the next section.

For this project, students can use PyCuda or CuPy for simplified GPU programming. Bonus points will be awarded to students who build their solutions with pure CUDA.

Project Choices

1. GGM Tree Construction

Build a pseudorandom function (PRF) tree based on the **Goldreich–Goldwasser–Micali (GGM)** construction, a classic method in theoretical cryptography for building secure PRFs from any length-doubling pseudorandom generator (PRG).

- PRF options: **AES, Keccak-f1600, Blake2s, or Spongent**.
 - Implement two of these, sign-up needed.
- Implement full tree expansion for varying seeds and depth.
- Discuss memory layout and PRF parallelization strategy.

Additional references:

- [Purdue Lecture Slides – GGM Construction \(Page 4\)](#)
- [MIT 6.875 Slides – GGM PRF Tree \(Page 16\)](#)
- [Keccak specifications summary](#)
- [BLAKE2 — fast secure hashing](#)
- [Spongent](#)

2. Merkle Tree with Post-Quantum Secure Hash

Build a **Merkle Tree** using a hash function that is post-quantum secure. Merkle trees are widely used in blockchain, integrity proofs, and digital signatures.

- Implement 2 of these hash functions: **SHA-3 (Keccak)**, **KangarooTwelve**, **Rescue**, or **Poseidon**, sign-up needed.
 - Hash functions must be implemented end-to-end (no pre-existing black-box hashing allowed)
- Implement tree generation from leaves up to the root.
- Implement Merkle proof generation/verification.

Additional references:

- [Merkle Tree](#)
- [SHA-3 - Wikipedia](#)
- [KangarooTwelve: fast hashing based on Keccak-p](#)
- [Rescue-Prime](#)
- [Poseidon Hash](#)

3. Modular Arithmetic Unit

Implement a modular arithmetic engine over a large prime field used in cryptographic operations.

- Support **modular multiplication**, **modular exponentiation**, and one of:
 - **Montgomery multiplication**,
 - **Barrett reduction**, or
 - **Modular inverse** (via Fermat or Extended Euclidean).

sign-up needed

- Apply your unit to a real cryptographic operation (e.g., scalar multiplication, RSA, or Number Theoretic Transform).

Additional references:

- [Montgomery modular multiplication](#)
- [Barrett reduction](#)
- [Modular multiplicative inverse](#)

4. Lightweight Block Cipher for Constrained Devices

Build and benchmark lightweight block ciphers designed for IoT/RFID where AES is too costly. A direct counterpoint to the AES discussion in lectures.

- Implement two of: PRESENT-80, GIFT-128, SIMON-64, SPECK-64, SKINNY-64. Sign-up needed.
- Implement encryption, decryption, and key schedule from the specification.
- Wrap each cipher in CTR and CBC modes, validate against test vectors.
- Quantitative comparison vs. an AES baseline: code size (or estimated gate count from a reference), cycles/byte, key-schedule cost.

Additional references:

- [PRESENT: An Ultra-Lightweight Block Cipher \(CHES 2007\)](#)
- [GIFT: A Small Present \(CHES 2017\)](#)
- [SIMON and SPECK \(NSA\)](#)
- [SKINNY family](#)

5. Hash-Based Digital Signatures

Build signature schemes whose security rests only on the hash function, a direct application of Merkle Tree's primitive and a key post-quantum signature family.

- Build a Lamport one-time signature as a warm-up.
- Implement two of: Winternitz OTS (W-OTS+), XMSS (stateful), LMS (stateful), or SPHINCS+ simplified variant (stateless). Sign-up needed.
- Implement KeyGen, Sign, Verify, and (where applicable) state management for stateful schemes.
- Quantify signature size, key size, and per-signature hash count vs. ECDSA at equivalent security level.

Additional references:

- [RFC 8391 — XMSS](#)
- [RFC 8554 — LMS](#)
- [SPHINCS+ submission](#)
- [Buchmann, Dahmen, Hülsing — XMSS practical hash-based signatures](#)

Sign up:

There are 26 3-student groups and two 2-student groups. Sign up sheet:

[+ ECE 268 project sign up sheet](#)

Sign-up Due Date: May 15, 2026

Compute:

<https://datahub.ucsd.edu/hub/spawn>

We will have datahub prepared for this course. Go to the above link, login, and choose your desired env (please allow some time for the datahub to be prepared, this might take a week or so after this document is shared with you).

Select Your Notebook Environment

☐	ECE268_SP25_A00 - Farinaz Koushanfar [SP25] ghcr.io/ucsd-ets/datascience-notebook:2025.2-stable (2 CPU, 8G RAM)
☐	ECE268_SP25_A00 - Farinaz Koushanfar [SP25] ghcr.io/ucsd-ets/scipy-ml-notebook:2025.2-stable (8 CPU, 16G RAM)
☒	ECE268_SP25_A00 - Farinaz Koushanfar [SP25] ghcr.io/ucsd-ets/scipy-ml-notebook:2025.2-stable (8 CPU, 16G RAM, 1 GPU)

Launch Environment

Presentation/Report:

- Jun 4, 2026 : Each group will give a 2.5-minute presentation due to the class size.
Please upload your slides to the Google Drive folder **in advance** of the presentation.
 - <https://drive.google.com/drive/folders/1p54icQgimTxvwobhy64ysg6KUkmaqnpO?usp=sharing>
 - Refer to canvas announcements for further information on the June 4 presentation.
- Jun 12, 2025 : Submit (1). a 10-minute presentation recording, (2). a fully operational CPU and GPU implementation github repository (with full history of contributions from

group members), and (3). a 5-page final report in double-column IEEE format. The report should discuss background, design choices, implementation, and results.

- Latex template for final report:
<https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhncsfqn>
- Submit the report on gradescope (**public github repository link must be included in your report**)
- Submit the recording on gradescope